Foram três entregas, e o recorte é a base das outras duas porque mexe com as fotos:

1. **Recorte de foto**: ao escolher a foto de **perfil** ou do **grupo**, abre uma tela escura onde você **arrasta e dá zoom** até enquadrar do jeito que quiser. O que você vê no quadro é exatamente o que fica salvo.
2. **Lista de membros**: ao tocar em *Membros do grupo* (ou em qualquer avatar da fileira).
3. **Perfil do amigo**: ao tocar num membro.

Decisões de design

- **O recorte é feito por nós, sem pacote novo.** Os pacotes de recorte exigem configurar o Android (e isso já deu trabalho várias vezes). Assim, não há nada para instalar.
- **Foto de perfil**: o quadro tem uma **guia circular**, e a parte de fora do círculo aparece escurecida. O app mostra a foto sempre em círculo, então você enxerga o que entra e o que sai.
- **Foto do grupo**: o quadro é um **quadrado de cantos arredondados**, com uma linha clara mostrando a parte que aparece nos ícones **redondos** (radar e chat).
- **Zoom**: o mínimo é "preencher o quadro", então nunca sobra borda vazia.
- **Tocar em "Você"** na lista de membros leva para a aba **Perfil**, e tocar num **amigo** abre o perfil dele.

Passo 1: Criar `lib/features/profile/recortar_foto_screen.dart`

dart

Dart (unsupported)

```dart
import 'dart:io';
import 'dart:math';
import 'dart:ui' as ui;
import 'package:flutter/foundation.dart' show kIsWeb;
import 'package:flutter/material.dart';
import 'package:flutter/rendering.dart';
import 'package:google_fonts/google_fonts.dart';

const _fundo = Color(0xFF081226);
const _blue = Color(0xFF0066FF);

// Lado da imagem final, em pixels
const int _saida = 800;

// O desenho da moldura de recorte
```

```dart
enum FormaRecorte { circulo, quadrado }

// Abre a tela de recorte.
// Devolve o caminho da imagem recortada, ou null se a pessoa
cancelou.
Future<String?> recortarImagem(
  BuildContext context,
  String caminho, {
  FormaRecorte forma = FormaRecorte.circulo,
  String titulo = 'Ajustar foto',
}) async {
  // No navegador não dá para gravar arquivos: usa a foto original
  if (kIsWeb) return caminho;

  return Navigator.of(context, rootNavigator: true).push<String>(
    MaterialPageRoute(
      fullscreenDialog: true,
      builder: (_) => RecortarFotoScreen(
        caminho: caminho,
        forma: forma,
        titulo: titulo,
      ),
    ),
  );
}

class RecortarFotoScreen extends StatefulWidget {
  final String caminho;
  final FormaRecorte forma;
  final String titulo;

  const RecortarFotoScreen({
    super.key,
    required this.caminho,
    required this.forma,
    required this.titulo,
```

```
  });

  @override
  State<RecortarFotoScreen> createState() =>
_RecortarFotoScreenState();
}

class _RecortarFotoScreenState extends State<RecortarFotoScreen> {
  // Marca a parte que será "fotografada" no final
  final _chave = GlobalKey();
  final _ctrl = TransformationController();

  ui.Image? _imagem;
  bool _erro = false;
  bool _salvando = false;
  bool _pronto = false; // já centralizou a foto no quadro?
  double _lado = 300; // lado do quadro, definido no build

  @override
  void initState() {
    super.initState();
    _carregar();
  }

  @override
  void dispose() {
    _ctrl.dispose();
    _imagem?.dispose();
    super.dispose();
  }

  Future<void> _carregar() async {
    try {
      final bytes = await File(widget.caminho).readAsBytes();
      final codec = await ui.instantiateImageCodec(bytes);
      final quadro = await codec.getNextFrame();
```

```
      codec.dispose();

      if (!mounted) {
        quadro.image.dispose();
        return;
      }
      setState(() => _imagem = quadro.image);
    } catch (e) {
      debugPrint('Erro ao abrir a imagem: $e');
      if (mounted) setState(() => _erro = true);
    }
  }

  // Coloca a foto centralizada, preenchendo o quadro
  void _centralizar() {
    final img = _imagem;
    if (img == null) return;
    final s = _lado / min(img.width, img.height);
    final w = img.width * s;
    final h = img.height * s;
    _ctrl.value = Matrix4.translationValues(-(w - _lado) / 2, -(h -
_lado) / 2, 0);
  }

  // Tira uma "foto" do quadro e salva em um arquivo
  Future<void> _confirmar() async {
    if (_salvando || _imagem == null) return;
    setState(() => _salvando = true);

    try {
      final fronteira =
          _chave.currentContext!.findRenderObject() as
RenderRepaintBoundary;
      final recorte = await fronteira.toImage(pixelRatio: _saida /
_lado);
```

```
      final dados = await recorte.toByteData(format:
ui.ImageByteFormat.png);
      recorte.dispose();
      if (dados == null) throw Exception('sem dados');

      final arquivo = File(

'${Directory.systemTemp.path}/gofind_foto_${DateTime.now().millisec
ondsSinceEpoch}.png',
      );
      await arquivo.writeAsBytes(dados.buffer.asUint8List());

      if (mounted) Navigator.pop(context, arquivo.path);
    } catch (e) {
      debugPrint('Erro ao recortar: $e');
      if (!mounted) return;
      setState(() => _salvando = false);
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text('Não foi possível recortar a foto. Tente de
novo.'),
        ),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    final tela = MediaQuery.of(context).size;
    final lado = min(tela.width - 40, tela.height * 0.52);
    _lado = lado;

    return Scaffold(
      backgroundColor: _fundo,
      body: SafeArea(
        child: Column(
```

```
          children: [
            // Cabeçalho: fechar, título e redefinir
            Padding(
              padding: const EdgeInsets.fromLTRB(8, 6, 8, 0),
              child: Row(
                children: [
                  IconButton(
                    onPressed:
                        _salvando ? null : () =>
Navigator.pop(context),
                    icon: const Icon(Icons.close, color:
Colors.white),
                    tooltip: 'Cancelar',
                  ),
                  Expanded(
                    child: Text(
                      widget.titulo,
                      style: GoogleFonts.comfortaa(
                        fontSize: 19,
                        fontWeight: FontWeight.w700,
                        color: Colors.white,
                      ),
                    ),
                  ),
                  TextButton(
                    onPressed: (_imagem == null || _salvando) ?
null : _centralizar,
                    child: Text(
                      'Redefinir',
                      style: GoogleFonts.comfortaa(
                        fontSize: 14,
                        fontWeight: FontWeight.w700,
                        color: Colors.white.withValues(alpha:
0.85),
                      ),
                    ),
```

```
                ),
              ],
            ),
          ),

          const Spacer(),

          Center(child: _quadro(lado)),

          const SizedBox(height: 18),
          Padding(
            padding: const EdgeInsets.symmetric(horizontal: 32),
            child: Text(
              'Arraste para mover e use dois dedos para aproximar
ou afastar.',
              textAlign: TextAlign.center,
              style: GoogleFonts.inter(
                fontSize: 14,
                color: Colors.white.withValues(alpha: 0.8),
                height: 1.35,
              ),
            ),
          ),
          if (widget.forma == FormaRecorte.quadrado) ...[
            const SizedBox(height: 6),
            Padding(
              padding: const EdgeInsets.symmetric(horizontal:
32),
              child: Text(
                'O círculo claro mostra o que aparece nos ícones
redondos.',
                textAlign: TextAlign.center,
                style: GoogleFonts.inter(
                  fontSize: 12.5,
                  color: Colors.white.withValues(alpha: 0.6),
                ),
```

```
              ),
            ),
          ],

          const Spacer(),

          // Botões
          Padding(
            padding: const EdgeInsets.fromLTRB(24, 0, 24, 20),
            child: Row(
              children: [
                Expanded(
                  child: SizedBox(
                    height: 50,
                    child: OutlinedButton(
                      onPressed:
                          _salvando ? null : () =>
Navigator.pop(context),
                      style: OutlinedButton.styleFrom(
                        side: BorderSide(
                          color: Colors.white.withValues(alpha:
0.6),
                        ),
                        shape: RoundedRectangleBorder(
                          borderRadius:
BorderRadius.circular(16),
                        ),
                      ),
                      child: Text(
                        'Cancelar',
                        style: GoogleFonts.comfortaa(
                          fontSize: 15,
                          fontWeight: FontWeight.w700,
                          color: Colors.white,
                        ),
                      ),
```

```
                    ),
                  ),
                ),
                const SizedBox(width: 12),
                Expanded(
                  child: SizedBox(
                    height: 50,
                    child: FilledButton(
                      onPressed: (_imagem == null || _salvando) ?
null : _confirmar,
                      style: FilledButton.styleFrom(
                        backgroundColor: _blue,
                        disabledBackgroundColor:
_blue.withValues(alpha: 0.5),
                        shape: RoundedRectangleBorder(
                          borderRadius:
BorderRadius.circular(16),
                        ),
                      ),
                      child: _salvando
                          ? const SizedBox(
                              width: 22,
                              height: 22,
                              child: CircularProgressIndicator(
                                strokeWidth: 2.5,
                                color: Colors.white,
                              ),
                            )
                          : Text(
                              'Usar foto',
                              style: GoogleFonts.comfortaa(
                                fontSize: 15,
                                fontWeight: FontWeight.w700,
                                color: Colors.white,
                              ),
                            ),
```

```
                    ),
                  ),
                ),
              ],
            ),
          ),
        ],
      ),
    ),
  );
}

// O quadro onde a foto é movida e ampliada
Widget _quadro(double lado) {
  if (_erro) {
    return SizedBox(
      width: lado,
      height: lado,
      child: Center(
        child: Text(
          'Não foi possível abrir esta foto.',
          style: GoogleFonts.inter(color: Colors.white70),
        ),
      ),
    );
  }

  final img = _imagem;
  if (img == null) {
    return SizedBox(
      width: lado,
      height: lado,
      child: const Center(
        child: CircularProgressIndicator(color: Colors.white),
      ),
    );
```

```
    }

    // Tamanho em que a foto "preenche" o quadro (o lado menor = o
quadro)
    final s = lado / min(img.width, img.height);
    final w = img.width * s;
    final h = img.height * s;

    // Na primeira vez, centraliza a foto
    if (!_pronto) {
      _pronto = true;
      WidgetsBinding.instance.addPostFrameCallback((_) {
        if (mounted) _centralizar();
      });
    }

    return SizedBox(
      width: lado,
      height: lado,
      child: Stack(
        children: [
          // Só o que está aqui dentro vira a foto final
          Positioned.fill(
            child: RepaintBoundary(
              key: _chave,
              child: InteractiveViewer(
                transformationController: _ctrl,
                constrained: false,
                minScale: 1,
                maxScale: 6,
                boundaryMargin: EdgeInsets.zero,
                child: RawImage(
                  image: img,
                  width: w,
                  height: h,
                  fit: BoxFit.fill,
```

```
                  filterQuality: FilterQuality.high,
                ),
              ),
            ),
          ),

          // Moldura (não faz parte da foto final)
          Positioned.fill(
            child: IgnorePointer(
              child: CustomPaint(painter:
_MolduraPainter(widget.forma)),
            ),
          ),
        ],
      ),
    );
  }
}

// Escurece o que fica fora da forma e desenha a borda
class _MolduraPainter extends CustomPainter {
  final FormaRecorte forma;
  _MolduraPainter(this.forma);

  @override
  void paint(Canvas canvas, Size size) {
    final r = Offset.zero & size;
    final escuro = Paint()..color = Colors.black.withValues(alpha:
0.55);
    final borda = Paint()
      ..color = Colors.white
      ..style = PaintingStyle.stroke
      ..strokeWidth = 2.5;

    if (forma == FormaRecorte.circulo) {
      final caminho = Path()
```

```
        ..fillType = PathFillType.evenOdd
        ..addRect(r)
        ..addOval(r);
      canvas.drawPath(caminho, escuro);
      canvas.drawOval(r.deflate(1.25), borda);
    } else {
      final rr = RRect.fromRectAndRadius(r, const
Radius.circular(24));
      final caminho = Path()
        ..fillType = PathFillType.evenOdd
        ..addRect(r)
        ..addRRect(rr);
      canvas.drawPath(caminho, escuro);
      canvas.drawRRect(rr.deflate(1.25), borda);

      // Guia: a parte que aparece nos ícones redondos
      final guia = Paint()
        ..color = Colors.white.withValues(alpha: 0.5)
        ..style = PaintingStyle.stroke
        ..strokeWidth = 1.5;
      canvas.drawCircle(r.center, size.width / 2, guia);
    }
  }

  @override
  bool shouldRepaint(_MolduraPainter antigo) => antigo.forma !=
forma;
}
```

Passo 2: `lib/features/profile/foto_perfil.dart` (substitua tudo)

Agora ele serve para perfil **e** grupo. Quem já usa `escolherFotoDoPerfil` (cadastro, editar perfil, Perfil) ganha o recorte **sem mudar nada**.

dart

Dart (unsupported)

```dart
import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';
import 'recortar_foto_screen.dart';

export 'recortar_foto_screen.dart' show FormaRecorte;

// Menu "Galeria / Câmera" + tela de recorte.
// Entrega o caminho da imagem final já recortada.
Future<void> escolherFoto(
  BuildContext context,
  void Function(String caminho) aoEscolher, {
  FormaRecorte forma = FormaRecorte.circulo,
  String titulo = 'Ajustar foto',
}) async {
  final origem = await showModalBottomSheet<ImageSource>(
    context: context,
    useRootNavigator: true,
    builder: (ctx) => SafeArea(
      child: Wrap(
        children: [
          ListTile(
            leading: const Icon(Icons.photo_library_outlined),
            title: const Text('Escolher da galeria'),
            onTap: () => Navigator.pop(ctx, ImageSource.gallery),
          ),
          ListTile(
            leading: const Icon(Icons.photo_camera_outlined),
            title: const Text('Tirar uma foto'),
            onTap: () => Navigator.pop(ctx, ImageSource.camera),
          ),
        ],
      ),
    ),
  );
  if (origem == null) return;
```

```dart
  // Pega a foto grande, para sobrar qualidade no recorte
  final imagem = await ImagePicker().pickImage(
    source: origem,
    maxWidth: 1600,
    imageQuality: 90,
  );
  if (imagem == null || !context.mounted) return;

  final recortada = await recortarImagem(
    context,
    imagem.path,
    forma: forma,
    titulo: titulo,
  );
  if (recortada != null) aoEscolher(recortada);
}

// Atalho usado nas telas de perfil
Future<void> escolherFotoDoPerfil(
  BuildContext context,
  void Function(String caminho) aoEscolher,
) =>
    escolherFoto(context, aoEscolher, titulo: 'Ajustar foto de
perfil');
```

Passo 3: Foto do grupo com recorte (`create_group_screen.dart`)

1. Imports. Apague esta linha:

dart

```
Dart (unsupported)
import 'package:image_picker/image_picker.dart';
```

e acrescente:

dart

```dart
import '../profile/foto_perfil.dart';
```

2. Apague esta linha (ela fica no começo de `_CreateGroupScreenState`):

dart

```dart
  final _picker = ImagePicker();
```

3. Apague os dois métodos `_escolherFoto` e `_mostrarOpcoesFoto` (o menu “Galeria / Câmera” antigo) e escreva no lugar um só:

dart

```dart
  // Escolhe a foto e deixa recortar em um quadrado
  void _mostrarOpcoesFoto() {
    escolherFoto(
      context,
      (caminho) => setState(() => _caminhoFoto = caminho),
      forma: FormaRecorte.quadrado,
      titulo: 'Ajustar foto do grupo',
    );
  }
```

O restante da tela não muda, porque o `_miniatura()` já chama `_mostrarOpcoesFoto`. O recorte também funciona em **Configurar grupo**, que é a mesma tela.

Passo 4: `lib/core/grupos_usuario.dart`: frase do perfil do amigo

Na classe `Membro`, troque do `class Membro {` até o fim do construtor (`});`) por:

dart

```dart
class Membro {
  final String nome;
  final bool ehVoce; // true = é o dono do celular
  final bool admin;
```

```
  final bool online;
  final String? caminhoFoto;

  // Frase do perfil (virá do próprio amigo, quando ele se
conectar)
  final String bio;

  // Onde o amigo está em relação a você. Virá da conexão entre os
celulares;
  // enquanto for null, o radar não desenha o amigo (só o lista).
  final double? distanciaM; // distância em metros
  final double? direcaoGraus; // 0 = norte, 90 = leste, 180 = sul,
270 = oeste

  const Membro({
    required this.nome,
    this.ehVoce = false,
    this.admin = false,
    this.online = false,
    this.caminhoFoto,
    this.bio = '',
    this.distanciaM,
    this.direcaoGraus,
  });
```

O getter `foto` que vem logo abaixo continua igual.

Passo 5: `lib/core/modo_teste.dart` (substitua tudo)

Só ganhou frases de perfil para os amigos de teste:

dart

```
Dart (unsupported)
import 'grupos_usuario.dart';

// Troque para true para testar o app com amigos fictícios:
```

```
// - o radar mostra 4 amigos
// - a lista de membros também mostra esses amigos
// - "Localizar Amigos" abre o radar mesmo com você sozinho no
grupo
// Lembre de voltar para false depois.
const bool amigosDeTeste = false;

// Troque para true para simular que você está andando até o amigo:
// na tela de seguir, a distância vai diminuindo sozinha até a
chegada.
// Lembre de voltar para false depois.
const bool simularAproximacao = false;

const List<Membro> amigosFicticios = [
  Membro(
    nome: 'João',
    online: true,
    distanciaM: 54,
    direcaoGraus: 254,
    bio: 'Sempre no meio da multidão',
  ),
  Membro(
    nome: 'Maria',
    online: true,
    distanciaM: 28,
    direcaoGraus: 316,
    bio: 'Chego já!',
  ),
  Membro(
    nome: 'Pedro',
    online: true,
    distanciaM: 63,
    direcaoGraus: 53,
    bio: 'Quem se perde paga a rodada',
  ),
  Membro(
```

```
    nome: 'Laura',
    distanciaM: 85,
    direcaoGraus: 130,
    bio: 'Aqui pelo show',
  ),
];
```

Passo 6: Criar `lib/core/widgets/foto_membro.dart`

A foto de um membro: a foto dele ou um círculo cinza com a inicial.

dart

```
Dart (unsupported)
import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';
import '../grupos_usuario.dart';

class FotoMembro extends StatelessWidget {
  final Membro membro;
  final double tamanho;
  final double borda; // 0 = sem borda

  const FotoMembro({
    super.key,
    required this.membro,
    required this.tamanho,
    this.borda = 0,
  });

  @override
  Widget build(BuildContext context) {
    final temFoto = membro.ehVoce || membro.caminhoFoto != null;

    return Container(
      width: tamanho,
      height: tamanho,
```

```
      alignment: Alignment.center,
      decoration: BoxDecoration(
        shape: BoxShape.circle,
        color: const Color(0xFFB8C4D9),
        border: borda > 0 ? Border.all(color: Colors.white, width:
borda) : null,
        image: temFoto
            ? DecorationImage(image: membro.foto, fit:
BoxFit.cover)
            : null,
      ),
      child: temFoto
          ? null
          : Text(
              membro.nome.isEmpty ? '?' :
membro.nome[0].toUpperCase(),
              style: GoogleFonts.comfortaa(
                fontSize: tamanho * 0.42,
                fontWeight: FontWeight.w700,
                color: Colors.white,
              ),
            ),
    );
  }
}
```

Passo 7: Criar `lib/features/groups/membros_screen.dart`

dart

Dart (unsupported)

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:google_fonts/google_fonts.dart';
import '../../core/grupos_usuario.dart';
import '../../core/modo_teste.dart';
import '../../core/navegacao_app.dart';
```

```dart
import '../../core/perfil_usuario.dart';
import '../../core/widgets/foto_membro.dart';
import 'perfil_amigo_screen.dart';

const _blue = Color(0xFF0066FF);
const _verde = Color(0xFF1FAF38);
const _cinza = Color(0xFF9AA5B8);

// O que acontece ao tocar num membro:
// você -> aba Perfil; amigo -> a tela de perfil dele
void abrirMembro(BuildContext context, Grupo grupo, Membro m) {
  if (m.ehVoce) {
    NavegacaoApp.pedidoDeAba.value = 3;
    return;
  }
  Navigator.push(
    context,
    MaterialPageRoute(
      builder: (_) => PerfilAmigoScreen(amigo: m, grupo: grupo),
    ),
  );
}

// Nome para mostrar: o seu nome de perfil, ou o nome do amigo
String _nomeDe(Membro m) {
  if (!m.ehVoce) return m.nome;
  return PerfilUsuario.nome.isEmpty ? 'Você' : PerfilUsuario.nome;
}

class MembrosScreen extends StatefulWidget {
  final Grupo grupo;
  const MembrosScreen({super.key, required this.grupo});

  @override
  State<MembrosScreen> createState() => _MembrosScreenState();
}
```

```dart
class _MembrosScreenState extends State<MembrosScreen> {
  final _busca = TextEditingController();

  @override
  void dispose() {
    _busca.dispose();
    super.dispose();
  }

  // Todos os membros (com os amigos de teste, se o modo teste
estiver ligado)
  List<Membro> get _todos => [
        ...widget.grupo.membros,
        if (amigosDeTeste) ...amigosFicticios,
      ];

  // Aplica a busca e ordena: você, administradores, online, depois
por nome
  List<Membro> _filtrados() {
    final termo = _busca.text.trim().toLowerCase();
    final lista = _todos
        .where((m) => termo.isEmpty ||
_nomeDe(m).toLowerCase().contains(termo))
        .toList();

    lista.sort((a, b) {
      if (a.ehVoce != b.ehVoce) return a.ehVoce ? -1 : 1;
      if (a.admin != b.admin) return a.admin ? -1 : 1;
      if (a.online != b.online) return a.online ? -1 : 1;
      return
_nomeDe(a).toLowerCase().compareTo(_nomeDe(b).toLowerCase());
    });
    return lista;
  }
```

```
  @override
  Widget build(BuildContext context) {
    final topo = MediaQuery.of(context).padding.top;

    return AnnotatedRegion<SystemUiOverlayStyle>(
      value: SystemUiOverlayStyle.light,
      // Redesenha se o grupo for editado
      child: ValueListenableBuilder<int>(
        valueListenable: GruposUsuario.versao,
        builder: (context, _, __) {
          final grupo = widget.grupo;
          final todos = _todos;
          final online = todos.where((m) => m.online).length;
          final membros = _filtrados();

          return Scaffold(
            backgroundColor: Colors.white,
            body: Column(
              children: [
                // Cabeçalho azul
                Container(
                  width: double.infinity,
                  color: _blue,
                  padding: EdgeInsets.fromLTRB(26, topo + 10, 26,
24),

                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Material(
                        color: Colors.white,
                        shape: const CircleBorder(),
                        child: InkWell(
                          customBorder: const CircleBorder(),
                          onTap: () => Navigator.pop(context),
                          child: const SizedBox(
                            width: 42,
```

```
                          height: 42,
                          child: Center(
                            child: Icon(Icons.arrow_back_ios_new,
                                size: 18, color: _blue),
                          ),
                        ),
                      ),
                    ),
                    const SizedBox(height: 8),
                    Row(
                      children: [
                        Expanded(
                          child: Column(
                            crossAxisAlignment:
CrossAxisAlignment.start,
                            children: [
                              Text(
                                'Membros',
                                style: GoogleFonts.comfortaa(
                                  fontSize: 32,
                                  fontWeight: FontWeight.w700,
                                  color: Colors.white,
                                ),
                              ),
                              const SizedBox(height: 2),
                              Text(
                                '${grupo.nome} · ${todos.length}
${todos.length == 1 ? 'membro' : 'membros'} · $online online',
                                maxLines: 2,
                                overflow: TextOverflow.ellipsis,
                                style: GoogleFonts.inter(
                                  fontSize: 15,
                                  color: Colors.white,
                                  height: 1.3,
                                ),
                              ),
```

```
                            ],
                          ),
                        ),
                        const SizedBox(width: 12),
                        Container(
                          width: 55,
                          height: 55,
                          decoration: BoxDecoration(
                            shape: BoxShape.circle,
                            border: Border.all(color:
Colors.white, width: 2),
                            image: DecorationImage(
                              image: grupo.foto,
                              fit: BoxFit.cover,
                            ),
                          ),
                        ),
                      ],
                    ),
                  ],
                ),
              ),

              // Busca
              Padding(
                padding: const EdgeInsets.fromLTRB(20, 16, 20,
4),
                child: TextField(
                  controller: _busca,
                  onChanged: (_) => setState(() {}),
                  textInputAction: TextInputAction.search,
                  style: GoogleFonts.inter(fontSize: 15),
                  decoration: InputDecoration(
                    hintText: 'Buscar membro...',
                    hintStyle:
```

```
                        GoogleFonts.inter(fontSize: 15, color:
_cinza),
                    prefixIcon: const Icon(Icons.search, color:
_blue),
                    isDense: true,
                    contentPadding: const
EdgeInsets.symmetric(vertical: 13),
                    enabledBorder: OutlineInputBorder(
                      borderRadius: BorderRadius.circular(16),
                      borderSide: const BorderSide(
                          color: Color(0xFFE3ECFF), width: 1.5),
                    ),
                    focusedBorder: OutlineInputBorder(
                      borderRadius: BorderRadius.circular(16),
                      borderSide: const BorderSide(color: _blue,
width: 1.5),
                    ),
                  ),
                ),
              ),

              // Lista
              Expanded(
                child: membros.isEmpty
                    ? Center(
                        child: Text(
                          'Nenhum membro encontrado',
                          style: GoogleFonts.comfortaa(
                            fontSize: 15,
                            fontWeight: FontWeight.w300,
                            color: const Color(0xFF6B6B6B),
                          ),
                        ),
                      )
                    : ListView.separated(
                        keyboardDismissBehavior:
```

```
ScrollViewKeyboardDismissBehavior.onDrag,
                          padding: const EdgeInsets.fromLTRB(20, 8,
20, 28),
                          itemCount: membros.length,
                          separatorBuilder: (_, __) => const
Padding(
                            padding: EdgeInsets.only(left: 68),
                            child: Divider(
                                height: 1, color:
Color(0xFFE5E5E5)),
                          ),
                          itemBuilder: (context, i) =>
_LinhaMembro(
                            membro: membros[i],
                            onTap: () =>
                                abrirMembro(context, grupo,
membros[i]),
                          ),
                        ),
                  ),
                ],
              ),
            );
          },
        ),
      );
  }
}

// Uma linha da lista de membros
class _LinhaMembro extends StatelessWidget {
  final Membro membro;
  final VoidCallback onTap;

  const _LinhaMembro({required this.membro, required this.onTap});
```

```
  @override
  Widget build(BuildContext context) {
    final d = membro.distanciaM;

    return InkWell(
      borderRadius: BorderRadius.circular(14),
      onTap: onTap,
      child: Padding(
        padding: const EdgeInsets.symmetric(vertical: 10),
        child: Row(
          children: [
            // Foto com a bolinha de online/offline
            SizedBox(
              width: 54,
              height: 54,
              child: Stack(
                clipBehavior: Clip.none,
                children: [
                  Positioned.fill(
                    child: FotoMembro(membro: membro, tamanho: 54),
                  ),
                  Positioned(
                    right: -1,
                    bottom: -1,
                    child: Container(
                      width: 15,
                      height: 15,
                      decoration: BoxDecoration(
                        color: membro.online ? _verde : _cinza,
                        shape: BoxShape.circle,
                        border: Border.all(color: Colors.white,
width: 2),
                      ),
                    ),
                  ),
```

```
      ],
    ),
  ),
  const SizedBox(width: 14),
  Expanded(
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Row(
          children: [
            Flexible(
              child: Text(
                _nomeDe(membro),
                maxLines: 1,
                overflow: TextOverflow.ellipsis,
                style: GoogleFonts.comfortaa(
                  fontSize: 18,
                  color: _blue,
                ),
              ),
            ),
            if (membro.ehVoce) ...[
              const SizedBox(width: 8),
              const _Selo('Você', cheio: false),
            ],
            if (membro.admin) ...[
              const SizedBox(width: 6),
              const _Selo('Administrador', cheio: true),
            ],
          ],
        ),
        const SizedBox(height: 3),
        Text(
          membro.ehVoce
              ? (membro.online ? 'Online' : 'Offline')
              : '${membro.online ? 'Online' : 'Offline'}'
```

```
                            '${d != null ? ' · ${d.round()} m' :
''}',
                    style: GoogleFonts.comfortaa(
                      fontSize: 12.5,
                      color: membro.online
                          ? _verde
                          : const Color(0xFF8A94A8),
                    ),
                  ),
                ],
              ),
            ),
            const Icon(Icons.chevron_right, color: _blue, size:
26),
          ],
        ),
      ),
    );
  }
}

// Etiqueta pequena: "Você" (contorno) ou "Administrador" (azul)
class _Selo extends StatelessWidget {
  final String texto;
  final bool cheio;

  const _Selo(this.texto, {required this.cheio});

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 8, vertical:
2),
      decoration: BoxDecoration(
        color: cheio ? _blue : Colors.transparent,
        borderRadius: BorderRadius.circular(10),
```

```
        border: Border.all(color: _blue, width: 1.2),
      ),
      child: Text(
        texto,
        style: GoogleFonts.comfortaa(
          fontSize: 10.5,
          fontWeight: FontWeight.w700,
          color: cheio ? Colors.white : _blue,
        ),
      ),
    );
  }
}
```

Passo 8: Criar `lib/features/groups/perfil_amigo_screen.dart`

dart

Dart (unsupported)

```dart
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:google_fonts/google_fonts.dart';
import '../../core/grupos_usuario.dart';
import '../../core/navegacao_app.dart';
import '../../core/widgets/foto_membro.dart';
import '../radar/seguir_screen.dart';

const _blue = Color(0xFF0066FF);
const _borda = Color(0xFFE3ECFF);
const _suave = Color(0xFFEAF2FF);
const _verde = Color(0xFF1FAF38);
const _laranja = Color(0xFFFF9800);
const _vermelho = Color(0xFFE53935);
const _cinza = Color(0xFF9AA5B8);

// Quão perto o amigo está (mesmos limites do radar)
(String, Color) _proximidade(Membro m) {
```

```
  final d = m.distanciaM;
  if (d == null) return ('Procurando sinal...', _cinza);
  if (d <= 30) return ('Muito Próximo', _verde);
  if (d <= 55) return ('Próximo', _verde);
  if (d <= 80) return ('Perto', _laranja);
  return ('Limite de alcance', _vermelho);
}

String _pontoCardeal(double graus) {
  const nomes = [
    'Norte', 'Nordeste', 'Leste', 'Sudeste',
    'Sul', 'Sudoeste', 'Oeste', 'Noroeste',
  ];
  final i = (((graus + 22.5) % 360) ~/ 45);
  return nomes[i];
}

class PerfilAmigoScreen extends StatelessWidget {
  final Membro amigo;
  final Grupo grupo;

  const PerfilAmigoScreen({super.key, required this.amigo, required
this.grupo});

  String get _primeiroNome => amigo.nome.split(' ').first;

  void _seguir(BuildContext context) {
    // Abre por cima da barra de baixo, como a tela de seguir
sempre faz
    Navigator.of(context, rootNavigator: true).push(
      MaterialPageRoute(builder: (_) => SeguirScreen(amigo:
amigo)),
    );
  }

  @override
```

```
  Widget build(BuildContext context) {
    final topo = MediaQuery.of(context).padding.top;
    final (proxTexto, proxCor) = _proximidade(amigo);
    final d = amigo.distanciaM;
    final dir = amigo.direcaoGraus;

    return AnnotatedRegion<SystemUiOverlayStyle>(
      value: SystemUiOverlayStyle.light,
      child: Scaffold(
        backgroundColor: Colors.white,
        body: ListView(
          padding: EdgeInsets.zero,
          children: [
            _cabecalho(context, topo),

            // O corpo branco sobe e cobre a base do cabeçalho
            Container(
              transform: Matrix4.translationValues(0, -26, 0),
              padding: const EdgeInsets.fromLTRB(16, 20, 16, 24),
              decoration: const BoxDecoration(
                color: Colors.white,
                borderRadius: BorderRadius.vertical(top:
Radius.circular(26)),
              ),
              child: Column(
                children: [
                  // Ações
                  Row(
                    children: [
                      Expanded(
                        child: _BotaoAcao(
                          icone: Icons.near_me,
                          texto: 'Ir até $_primeiroNome',
                          preenchido: true,
                          onTap: () => _seguir(context),
                        ),
```

```
                    ),
                    const SizedBox(width: 12),
                    Expanded(
                      child: _BotaoAcao(
                        icone: Icons.radar,
                        texto: 'Ver no radar',
                        preenchido: false,
                        onTap: () =>
NavegacaoApp.abrirRadar(grupo),
                      ),
                    ),
                  ],
                ),
                const SizedBox(height: 18),

                // Localização
                _Cartao(
                  titulo: 'Localização',
                  linhas: [
                    _LinhaInfo(
                      icone: Icons.social_distance,
                      titulo: 'Distância',
                      valor: d == null
                          ? 'Procurando sinal...'
                          : '${d.round()} m',
                    ),
                    _LinhaInfo(
                      icone: Icons.explore,
                      titulo: 'Direção',
                      valor: dir == null
                          ? 'Calculando...'
                          : '${_pontoCardeal(dir)}
(${dir.round()}°)',
                    ),
                    _LinhaInfo(
                      icone: Icons.sensors,
```

```
                      titulo: 'Sinal',
                      valor: proxTexto,
                      corPonto: proxCor,
                    ),
                  ],
                ),
                const SizedBox(height: 14),

                // No grupo
                _Cartao(
                  titulo: 'No grupo',
                  linhas: [
                    _LinhaInfo(
                      icone: Icons.groups,
                      titulo: 'Grupo',
                      valor: grupo.nome,
                    ),
                    _LinhaInfo(
                      icone: amigo.admin
                          ? Icons.workspace_premium
                          : Icons.person_outline,
                      titulo: 'Função',
                      valor: amigo.admin ? 'Administrador' :
'Participante',
                    ),
                    _LinhaInfo(
                      icone: Icons.wifi_tethering,
                      titulo: 'Status',
                      valor: amigo.online ? 'Online' : 'Offline',
                      corPonto: amigo.online ? _verde : _cinza,
                    ),
                  ],
                ),
                const SizedBox(height: 16),

                Padding(
```

```
                    padding: const EdgeInsets.symmetric(horizontal:
8),
                    child: Text(
                      'As informações de localização aparecem
quando seu amigo está ao alcance do seu celular.',
                      textAlign: TextAlign.center,
                      style: GoogleFonts.inter(
                        fontSize: 12.5,
                        color: const Color(0xFF8A94A8),
                        height: 1.35,
                      ),
                    ),
                  ),
                ],
              ),
            ),
          ],
        ),
      ),
    );
  }

  // Faixa azul com foto, nome, status e frase
  Widget _cabecalho(BuildContext context, double topo) {
    return ClipRect(
      child: Container(
        width: double.infinity,
        color: _blue,
        child: Stack(
          children: [
            // Círculos decorativos
            Positioned(
              left: -70,
              top: 150,
              child: _Bolha(230),
            ),
```

```
Positioned(
  right: -100,
  top: -30,
  child: _Bolha(300),
),

Padding(
  padding: EdgeInsets.fromLTRB(24, topo + 18, 24, 52),
  child: Column(
    children: [
      // Foto (com a coroa se for administrador)
      SizedBox(
        width: 124,
        height: 124,
        child: Stack(
          clipBehavior: Clip.none,
          children: [
            Positioned.fill(
              child: FotoMembro(
                membro: amigo,
                tamanho: 124,
                borda: 4,
              ),
            ),
            if (amigo.admin)
              Positioned(
                right: 2,
                bottom: 4,
                child: Container(
                  width: 34,
                  height: 34,
                  decoration: BoxDecoration(
                    color: _blue,
                    shape: BoxShape.circle,
                    border:
```

```
                                    Border.all(color: Colors.white,
width: 3),
                                ),
                                child: const
Icon(Icons.workspace_premium,
                                    size: 18, color: Colors.white),
                              ),
                            ),
                          ],
                        ),
                      ),
                      const SizedBox(height: 12),
                      Text(
                        amigo.nome,
                        maxLines: 1,
                        overflow: TextOverflow.ellipsis,
                        style: GoogleFonts.comfortaa(
                          fontSize: 28,
                          fontWeight: FontWeight.w700,
                          color: Colors.white,
                        ),
                      ),
                      const SizedBox(height: 6),
                      Container(
                        padding:
                            const EdgeInsets.symmetric(horizontal: 14,
vertical: 5),
                        decoration: BoxDecoration(
                          color: Colors.white.withValues(alpha: 0.16),
                          borderRadius: BorderRadius.circular(16),
                        ),
                        child: Row(
                          mainAxisSize: MainAxisSize.min,
                          children: [
                            Container(
                              width: 9,
```

```
                    height: 9,
                    decoration: BoxDecoration(
                      shape: BoxShape.circle,
                      color: amigo.online
                          ? const Color(0xFF3CE04B)
                          : Colors.white54,
                    ),
                  ),
                  const SizedBox(width: 7),
                  Text(
                    amigo.online ? 'Online' : 'Offline',
                    style: GoogleFonts.inter(
                      fontSize: 14,
                      color: Colors.white,
                    ),
                  ),
                ],
              ),
            ),
            if (amigo.bio.isNotEmpty) ...[
              const SizedBox(height: 10),
              Text(
                amigo.bio,
                textAlign: TextAlign.center,
                maxLines: 2,
                overflow: TextOverflow.ellipsis,
                style: GoogleFonts.inter(
                  fontSize: 15,
                  color: Colors.white,
                ),
              ),
            ],
          ],
        ),
      ),
```

```
            // Voltar
            Positioned(
              top: topo + 10,
              left: 15,
              child: Material(
                color: Colors.white,
                shape: const CircleBorder(),
                child: InkWell(
                  customBorder: const CircleBorder(),
                  onTap: () => Navigator.pop(context),
                  child: const SizedBox(
                    width: 42,
                    height: 42,
                    child: Center(
                      child: Icon(Icons.arrow_back_ios_new,
                          size: 18, color: _blue),
                    ),
                  ),
                ),
              ),
            ),
          ],
        ),
      ),
    );
  }
}

// Círculo claro decorativo do cabeçalho
class _Bolha extends StatelessWidget {
  final double tamanho;
  const _Bolha(this.tamanho);

  @override
  Widget build(BuildContext context) {
    return Container(
```

```
      width: tamanho,
      height: tamanho,
      decoration: BoxDecoration(
        shape: BoxShape.circle,
        color: Colors.white.withValues(alpha: 0.06),
      ),
    );
  }
}

// Botão grande: "Ir até ..." (azul) e "Ver no radar" (contorno)
class _BotaoAcao extends StatelessWidget {
  final IconData icone;
  final String texto;
  final bool preenchido;
  final VoidCallback onTap;

  const _BotaoAcao({
    required this.icone,
    required this.texto,
    required this.preenchido,
    required this.onTap,
  });

  @override
  Widget build(BuildContext context) {
    final cor = preenchido ? Colors.white : _blue;

    return Material(
      color: preenchido ? _blue : Colors.white,
      borderRadius: BorderRadius.circular(16),
      child: InkWell(
        borderRadius: BorderRadius.circular(16),
        onTap: onTap,
        child: Container(
          height: 54,
```

```
          padding: const EdgeInsets.symmetric(horizontal: 10),
          decoration: BoxDecoration(
            borderRadius: BorderRadius.circular(16),
            border: preenchido ? null : Border.all(color: _blue,
width: 1.8),
          ),
          child: Row(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Icon(icone, color: cor, size: 22),
              const SizedBox(width: 8),
              Flexible(
                child: FittedBox(
                  fit: BoxFit.scaleDown,
                  child: Text(
                    texto,
                    style: GoogleFonts.comfortaa(
                      fontSize: 15,
                      fontWeight: FontWeight.w700,
                      color: cor,
                    ),
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

// Cartão com título e linhas de informação
class _Cartao extends StatelessWidget {
  final String titulo;
  final List<Widget> linhas;
```

```dart
  const _Cartao({required this.titulo, required this.linhas});

  @override
  Widget build(BuildContext context) {
    return Container(
      width: double.infinity,
      padding: const EdgeInsets.fromLTRB(16, 14, 16, 8),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(18),
        border: Border.all(color: _borda, width: 1.5),
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            titulo,
            style: GoogleFonts.comfortaa(
              fontSize: 17,
              fontWeight: FontWeight.w700,
              color: _blue,
            ),
          ),
          const SizedBox(height: 6),
          for (var i = 0; i < linhas.length; i++) ...[
            linhas[i],
            if (i < linhas.length - 1)
              const Divider(height: 1, color: Color(0xFFE5E5E5)),
          ],
        ],
      ),
    );
  }
}
```

```dart
// Linha "ícone + título ... valor"
class _LinhaInfo extends StatelessWidget {
  final IconData icone;
  final String titulo;
  final String valor;
  final Color? corPonto;

  const _LinhaInfo({
    required this.icone,
    required this.titulo,
    required this.valor,
    this.corPonto,
  });

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 11),
      child: Row(
        children: [
          Container(
            width: 36,
            height: 36,
            decoration: const BoxDecoration(
              color: _suave,
              shape: BoxShape.circle,
            ),
            child: Icon(icone, color: _blue, size: 19),
          ),
          const SizedBox(width: 12),
          Text(
            titulo,
            style: GoogleFonts.comfortaa(
              fontSize: 15,
              color: const Color(0xFF1B3A73),
            ),
```

```
        ),
        const SizedBox(width: 12),
        Expanded(
          child: Row(
            mainAxisAlignment: MainAxisAlignment.end,
            children: [
              if (corPonto != null) ...[
                Container(
                  width: 9,
                  height: 9,
                  decoration: BoxDecoration(
                    color: corPonto,
                    shape: BoxShape.circle,
                  ),
                ),
                const SizedBox(width: 6),
              ],
              Flexible(
                child: Text(
                  valor,
                  maxLines: 1,
                  overflow: TextOverflow.ellipsis,
                  textAlign: TextAlign.end,
                  style: GoogleFonts.comfortaa(
                    fontSize: 14.5,
                    fontWeight: FontWeight.w700,
                    color: const Color(0xFF4A5875),
                  ),
                ),
              ),
            ],
          ),
        ),
      ],
    ),
  );
```

```
  }
}
```

Passo 9: Ligar na tela do grupo (`group_screen.dart`)

1. Import. No topo, junto dos outros:

dart

```
Dart (unsupported)
import 'membros_screen.dart';
```

2. Título "Membros do grupo". Procure:

dart

```
Dart (unsupported)
                    onTap: () => _emBreve(context, 'Lista de
membros'),
```

e troque por:

dart

```
Dart (unsupported)
                    onTap: () => Navigator.push(
                      context,
                      MaterialPageRoute(
                        builder: (_) => MembrosScreen(grupo:
grupo),
                      ),
                    ),
```

3. Avatares da fileira tocáveis. Procure:

dart

```
Dart (unsupported)
                        for (final m in grupo.membros)
_MembroItem(m),
```

e troque por:

dart

```
Dart (unsupported)
                        for (final m in grupo.membros)
                          GestureDetector(
                            onTap: () => abrirMembro(context,
grupo, m),
                            child: _MembroItem(m),
                          ),
```

Como testar

1. Salve tudo e **pare o app e rode de novo** (`q`, depois `flutter run -d RQGL301KN8P`). Não há pacote novo para instalar.

Recorte da foto de perfil
2. **Perfil** → toque na foto ou na câmera → **Escolher da galeria**. Abre a tela escura **"Ajustar foto de perfil"**, com a foto preenchendo um quadrado e o **círculo** destacado (o resto escurecido).
3. **Arraste** para mover e use **dois dedos** para aproximar e afastar. A foto não deixa o quadro ficar vazio. **Redefinir** centraliza de novo.
4. **Usar foto**: o botão mostra um carregando por um instante e a foto aparece recortada no Perfil, na Home, em Grupos e no Radar. **Cancelar** ou o **X** não muda nada.
5. O mesmo recorte aparece no **cadastro** (primeira tela de perfil) e em **Editar perfil**.

Recorte da foto do grupo
6. **Criar grupo** (ou **Configurar**) → toque na miniatura → o recorte abre com um **quadrado de cantos arredondados** e um **círculo claro** dentro, que mostra o que aparece nos ícones redondos (chat e radar).

Membros
7. Abra um grupo e toque em **Membros do grupo** (título ou seta): abre a lista com o cabeçalho azul "Membros", a contagem e a busca. Você aparece primeiro, com os selos **Você** e **Administrador**.
8. Com `amigosDeTeste = true` (em `lib/core/modo_teste.dart`, e **R**): a lista mostra também João, Maria, Pedro e Laura, com **Online/Offline** e a distância. Digite "ma" na busca:

sobra só a Maria.
9. Toque num **amigo**: abre o perfil, com foto, nome, status, a frase e os cartões **Localização** e **No grupo**.

- **Ir até João** abre a tela de seguir (a seta).
- **Ver no radar** abre a aba Radar já nesse grupo.

10. Toque em **“Você”** (na lista ou na fileira de avatares da tela do grupo): vai para a aba **Perfil**.
11. Na tela do grupo, toque num avatar da **fileira de membros**: também abre o perfil do amigo (com o modo de teste, só a sua foto aparece nessa fileira, porque ela mostra os membros reais).

Decisões e limites

1. **A foto final é sempre quadrada (800 × 800).** No perfil, o círculo é só a guia: o app mostra a foto em círculo, e o que está nos cantos fica de fora. É o que você vê escurecido.
2. **A foto recortada é salva na pasta temporária do Android**, igual antes: pode ser apagada pelo sistema. Quando fizermos o **Hive**, a foto é copiada para dentro do app.
3. **No navegador (Brave) o recorte é ignorado**, porque ele depende de gravar arquivo. Teste no celular.
4. **Perfil do amigo**: a **frase**, a **foto** e o **status** dos amigos reais virão deles mesmos quando a conexão entre os celulares existir. Hoje só os amigos de teste têm dados.
5. **Ações do administrador** (remover membro, promover a administrador, aprovar quem pede para entrar) **não entram agora**, porque dependem da conexão entre os celulares e botões que não funcionam enganam. Eles entram na lista e no perfil quando esse trecho existir.
6. **Duas coisas ficaram duplicadas de propósito** (os limites de “Muito Próximo / Próximo / Perto” aparecem no radar e no perfil do amigo). Quando mexermos de novo nesses limites, vale juntar num arquivo só.